
Curriculum Ordering and Out-of-Distribution Generalization in Proximal Policy Optimization

Keisuke Nishioka

github.com/keisuke58/rl-curriculum-ood

Team: Single-student project (Keisuke Nishioka, solo).

1 Motivation

Curriculum learning—ordering training from easy to hard—is widely believed to improve convergence in reinforcement learning, yet existing work almost exclusively evaluates on the *same* environments used for training. Whether a carefully ordered curriculum also helps an agent *generalize* to novel, unseen environments is an open question with practical importance: a robot or autonomous agent must ultimately act outside its training distribution. This project asks whether curriculum ordering affects out-of-distribution (OOD) generalization in MiniGrid, and whether the curriculum condition that maximizes training-environment performance is the same one that maximizes transfer. If the two objectives conflict, this reveals a concrete curriculum-design trade-off not previously characterized in the deep RL generalization literature.

2 Related Topics

Curriculum Learning (Bengio et al., 2009). The foundational paper proposes ordering training examples from easy to hard to accelerate learning. We test whether this principle holds for *generalization* as well as convergence speed in the RL setting.

Generalization in Deep RL (Kirk et al., 2023 — Week 11). The survey identifies generalization to unseen environments as a central open problem. We adopt their zero-shot OOD evaluation protocol and use MiniGrid’s procedural structure to create held-out test environments.

Proximal Policy Optimization (Schulman et al., 2017 — Weeks 5–6). PPO is the baseline algorithm throughout. Its clipped surrogate objective and on-policy nature make it a standard choice for MiniGrid-scale experiments and a direct extension of the lecture example.

3 Idea

We train a PPO agent under five curriculum conditions that differ only in the *ordering* of three MiniGrid environments (Easy → Medium → Hard, Reverse, Random, Hard-only, Mixed multi-task). After training, every agent is evaluated zero-shot on two held-out environments it has never seen. The key quantity is the *generalization gap*: does the condition with the highest training return also achieve the highest OOD success rate, or does curriculum-induced overfitting to the training distribution hurt transfer?

Algorithm 1 Threshold-based curriculum switching for Progressive and Reverse conditions.

Require: ordered environment list $[e_1, e_2, e_3]$, threshold $\tau = 0.70$, window $W = 20$

```

 $i \leftarrow 1$ , successes  $\leftarrow []$ 
while steps  $< 10^6$  do
    Collect episode in  $e_i$  with PPO; append outcome to successes
    if mean(last  $W$  of successes)  $\geq \tau$  and  $i < 3$  then
         $i \leftarrow i + 1$  {advance to next environment}
    end if
end while
return policy  $\pi_\theta$ 

```

The OOD evaluation metric is defined as:

$$\text{OOD-SR}(\pi) = \frac{1}{|E_{\text{test}}|} \sum_{e \in E_{\text{test}}} \frac{1}{50} \sum_{k=1}^{50} \mathbf{1}[R_k(e) > 0] \quad (1)$$

where $E_{\text{test}} = \{\text{DoorKey-5x5}, \text{MultiRoom-N2-S4}\}$ and $R_k(e)$ is the terminal reward of episode k .

4 Experiments

Environments & Metrics Three MiniGrid environments form the training set (Empty-8x8, FourRooms, KeyCorridorS3R1, ordered easy to hard); two structurally distinct environments (DoorKey-5x5, MultiRoom-N2-S4) are held out for zero-shot evaluation only. We measure (1) mean in-distribution return over the last 10^5 training steps, (2) OOD success rate (Eq. 1), and (3) area under the learning curve as a sample-efficiency proxy. Statistical significance between conditions uses Kruskal–Wallis followed by pairwise Mann–Whitney U tests with Bonferroni correction ($\alpha = 0.05$).

Experimental Scope Five curriculum conditions $\times$ 10 seeds = 50 main runs, each for 10^6 environment steps. Two ablations on curriculum switching threshold (60% vs. 80% success rate) add 20 runs. Total: **70 runs**, all with identical PPO hyperparameters ($\text{lr} = 3 \times 10^{-4}$, $n_{\text{steps}} = 1024$, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, entropy coeff. = 0.01).

Estimated Computational Load One run takes approximately 25–30 minutes on a single CPU core for MiniGrid-FourRooms at 10^6 steps. With 10 parallel processes on the available cluster (4 $\times$ RTX 4090 servers), all 70 runs complete in roughly 4–5 hours wall-clock time. Memory footprint per process is under 2 GB; no GPU is required for MiniGrid (CPU-bound simulation). Analysis scripts (analysis/) run in under 5 minutes on any laptop.

5 Timeline

- **Research:** literature review and hypothesis formulation — complete
- **Implementation:** PPO training loop, curriculum switching, OOD evaluation scripts — complete (train.py, curriculum.py, evaluate.py)
- **Experiments:** 70 distributed runs on cluster — 2026-07-01 to 07-07
- **Analysis:** result aggregation, statistical tests, figures — 2026-07-08 to 07-12
- **Reporting:** Report (≤ 4 pages, NeurIPS-style adr1 format) + Poster (PDF) — 2026-07-15 to 08-20; submission 2026-08-23 via Stud.IP